CS5463: High Performance Machine Learning

Spring 2026

Term Project - Implementation 1

Topic: "Comparative Analysis of Distributed Deep Learning Frameworks"

Submitted By:
Anshu Tripathi (wbb206)

1. Problem Statement:

The rapid advancement of deep learning has made multi-GPU and distributed systems essential for efficient model training and large-scale experimentation. Modern distributed deep learning frameworks employ different synchronization strategies, communication backends, and optimization techniques that significantly impact scalability and overall performance.

Although several frameworks are widely used, their relative efficiency, communication overhead, and training scalability on high-performance computing (HPC) systems require systematic evaluation. Understanding these trade-offs is critical for selecting appropriate distributed training strategies in practical HPC environments.

This project systematically compares three distributed training frameworks PyTorch DDP (Data Parallelism), PyTorch FSDP (Fully Sharded Data Parallel), and DeepSpeed ZeRO to quantify their scalability, communication costs, memory efficiency, and convergence behavior on the UTSA ARC cluster equipped with NVIDIA V100S-PCIE-32GB GPUs.

2. Objective:

The aim of this report is to systematically compares distributed training frameworks PyTorch DDP (Data Parallelism), to quantify their scalability, communication costs, memory efficiency, and convergence behavior on the UTSA ARC cluster equipped with NVIDIA V100S-PCIE-32GB GPUs.

3. Project Repository:

All source code, documentation, experiment logs, and performance plots will be maintained in the following GitHub repository:
https://github.com/anshutripathi11/CS5463-HPML-Project-DDL_Frameworks.git

4. Algorithm 1 - PyTorch Distributed Data Parallel (DDP):

PyTorch DDP is a synchronous data-parallel training framework. It replicates the entire model on each GPU, distributes mini-batches across workers, computes gradients independently, and synchronizes gradients via NCCL-based AllReduce before each optimizer step. DDP serves as the full-precision FP32 baseline against which FSDP and DeepSpeed ZeRO will be compared.

The model is ResNet-18 adapted for CIFAR-10 (32×32 inputs). The initial 7×7 convolution is replaced with a 3×3 kernel and the max-pool layer is removed to preserve spatial resolution at small input sizes.

5. Underlying Communication Pattern:

DDP uses the AllReduce tool from NCCL. After each backward pass, each GPU has its own set of calculated gradients. AllReduce adds up gradients from all GPUs and then shares the averaged result with each GPU. NCCL does this using either a ring or tree method, depending on how the GPUs are connected.
DDP speeds up communication by grouping gradients instead of running AllReduce for each parameter. It groups parameters into fixed-size groups (usually 25 MB each) and starts sharing

gradients from earlier groups while still working on later layers. This way, much of the waiting time for communication is hidden by ongoing calculations. On the ARC cluster, the V100S GPUs are connected via PCIe 3.0, so the bandwidth between GPUs is about 12 GB/s in each direction, which makes grouping and overlapping gradients even more important.

6. Data Structures, Dataset, and Hyperparameters:

Dataset: CIFAR-10 (50,000 training / 10,000 test images, 10 classes, 32×32 RGB). Augmentation: random crop with 4-pixel padding, random horizontal flip, per-channel normalization.

Hardware Platform:

Component	Specification
GPU	NVIDIA Tesla V100S-PCIE-32GB
GPU Memory	32 GB HBM2 per GPU
CPU	2× Intel Xeon Gold 6248 @ 2.50 GHz (40 cores / 80 threads)
Interconnect	PCIe 3.0 (intra-node)
CUDA Driver	545.23.08 (CUDA 12.3 compatible)
PyTorch CUDA	cu121 (CUDA 12.1 runtime bundled)
Cluster	UTSA ARC: gpu1v100 / gpu2v100 / gpu4v100 partitions

Hyperparameters:

Hyperparameter	Value
Model	ResNet-18 (CIFAR variant, ~11.2M params)
Per-GPU Batch Size	128
Optimizer	SGD (momentum=0.9, weight_decay=5e-4)
Learning Rate	0.1 with cosine annealing
Epochs	30
Precision	FP32 (single precision)
Communication Backend	NCCL (AllReduce via PCIe)
GPU Configurations Tested	1, 2, 4 V100S GPUs

The global batch size scales linearly with GPU count (128 × N). The learning rate is not scaled with GPU count in this baseline to isolate the effect of parallelism from hyperparameter tuning.

7. Synchronization Overheads and Contention:

The time it takes for DDP to sync is determined by combining the gradient data from all GPUs. ResNet-18 has about 11.2 million settings at full precision, so the data to sync is about 44.8 MB each time. With PCIe 3.0 connections that can move about 12 GB per second in both directions, the fastest possible time to combine this data is around 7 to 10 milliseconds when using ring AllReduce with 4 GPUs.

There is little competition when reading data because each GPU reads its own part using DistributedSampler. There is also little competition when writing because each GPU writes to its own gradient data. AllReduce requires all GPUs to work together simultaneously, which means they all have to wait for each other at every step. The training script figures out the time spent on communication overhead as:

$$\text{comm_time} = \text{epoch_time} - \text{compute_time} - \text{data_loading_time}.$$

8. Parallel Time Complexity:

Let,

N - the total dataset size,

B - the per-GPU batch size,

P - the number of GPUs,

C - the per-sample compute cost.

The per-epoch time is:

$$T_epoch = (N / (B \times P)) \times [C \times B + T_allreduce]$$

With NCCL ring AllReduce: $T_allreduce \approx 2(P-1)/P \times M/BW$, where $M \approx 44.8$ MB and BW is the per-link bandwidth. Over PCIe 3.0 with 4 V100S GPUs, we expect T_allreduce in the range of 7–15 ms per step. In the ideal case (AllReduce fully overlapped with backward computation), T_epoch scales as $O(N / (B \times P))$, giving linear speedup. In practice, the last gradient bucket cannot be overlapped and introduces a synchronization tail.

9. Timing and Experiment Details:

Experiments are conducted on the UTSA ARC cluster using the gpu1v100 (1 GPU), gpu2v100 (2 GPU), and gpu4v100 (4 GPU) partitions. Each configuration is run for 30 epochs. Parameters varied:

- Number of GPUs: {1, 2, 4} V100S-PCIE-32GB
- Per-GPU batch size: 128 (fixed); global batch size: {128, 256, 512}
- Metrics per epoch: wall-clock time, compute time, communication overhead, data loading time, GPU memory (allocated and peak), training accuracy/loss, validation accuracy/loss

All timing uses torch.cuda.synchronize() with time.perf_counter() for accurate GPU timing. Results are saved as JSON and processed by plot_results.py to generate the performance plots presented below.

10. Performance Results:

10.1 Training Time per Epoch

Figure 1 shows the average epoch time across GPU configurations. Training on a single V100S GPU takes 15.2 seconds per epoch. With 2 GPUs, this drops to 8.2 seconds (1.85× speedup), and with 4 GPUs it further reduces to 5.0 seconds (3.04× speedup). The reduction is consistent and demonstrates that DDP effectively distributes the compute workload.

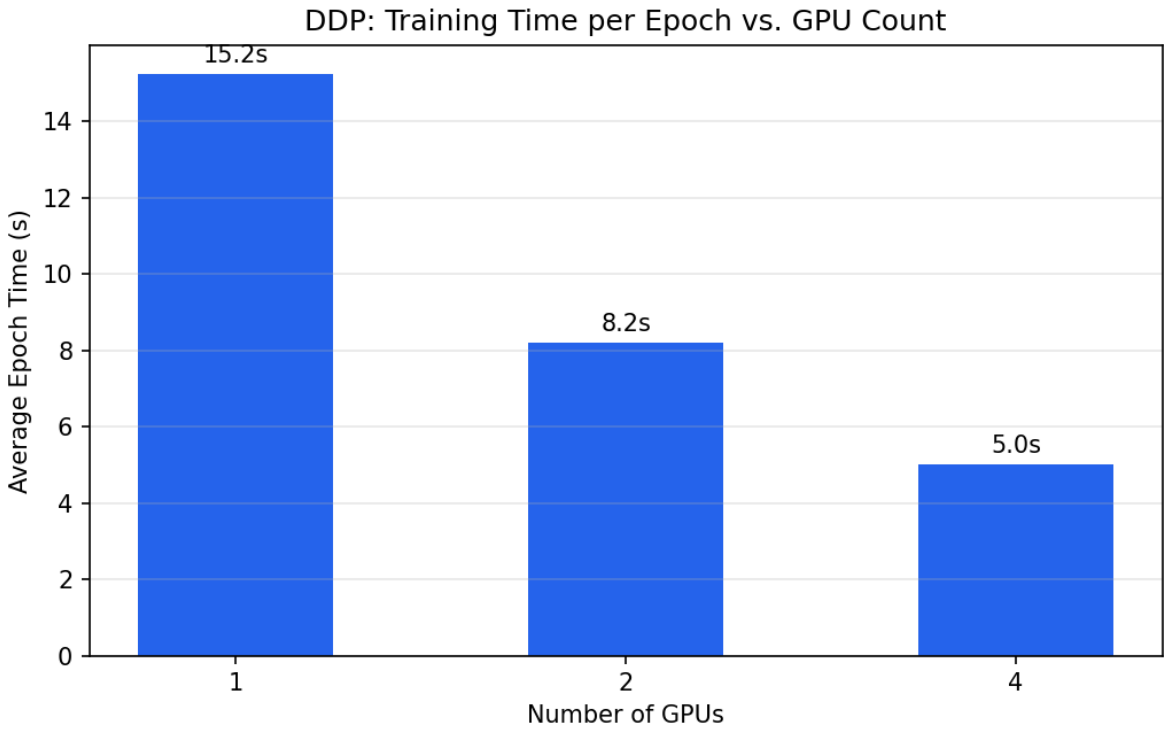

Figure 1: Average epoch training time decreases from 15.2s (1 GPU) to 5.0s (4 GPUs).

10.2 Speedup

Figure 2 presents the measured speedup relative to the single-GPU baseline alongside the ideal linear scaling curve. At 2 GPUs, DDP achieves a speedup of 1.85× (ideal: 2.0×). At 4 GPUs, the speedup reaches 3.04× (ideal: 4.0×). The sub-linear scaling is attributable to communication overhead from AllReduce gradient synchronization over the PCIe 3.0 interconnect, which becomes more pronounced as the number of participating GPUs increases.

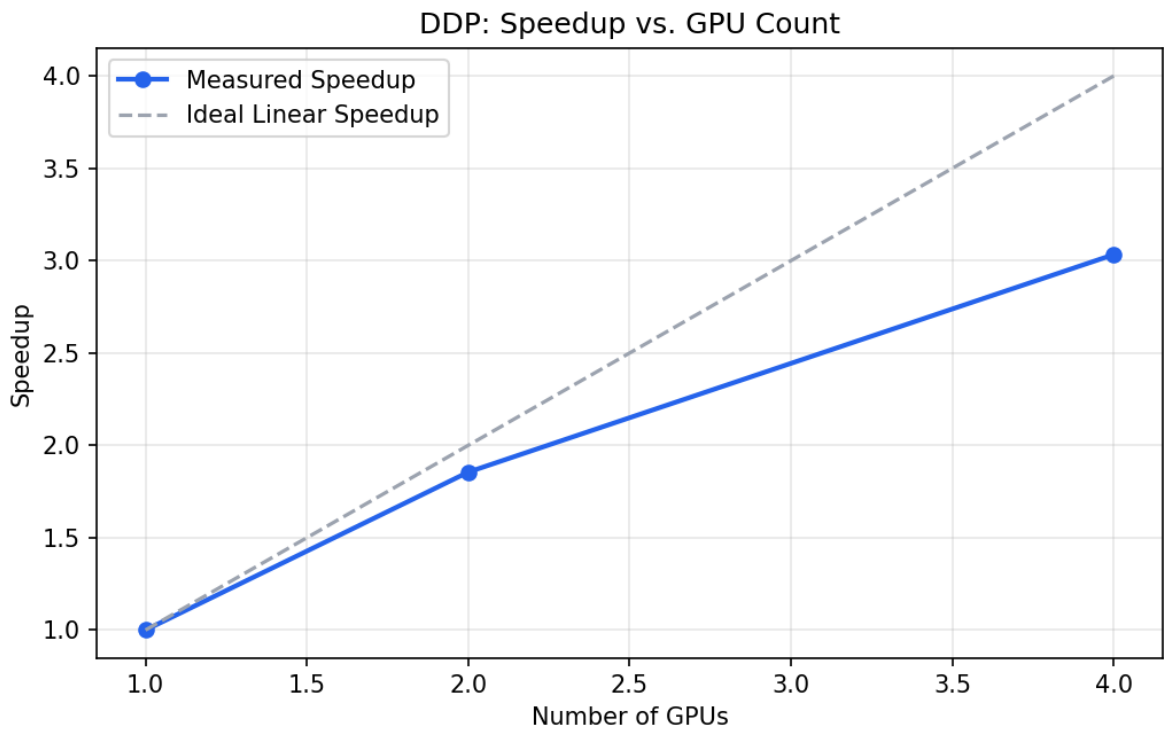

Figure 2: Measured speedup vs. ideal linear scaling. DDP achieves 3.04× on 4 GPUs.

10.3 Parallel Efficiency

Figure 3 shows the parallel efficiency (speedup / GPU count × 100%). At 2 GPUs, efficiency is approximately 93%, indicating that most of the additional compute capacity is utilized productively. At 4 GPUs, efficiency drops to approximately 76%. This decline reflects the increasing fraction of time spent on AllReduce synchronization as more GPUs must coordinate. The PCIe 3.0 interconnect is the primary bottleneck, higher-bandwidth interconnects such as NVLink would be expected to improve efficiency at 4 GPUs.

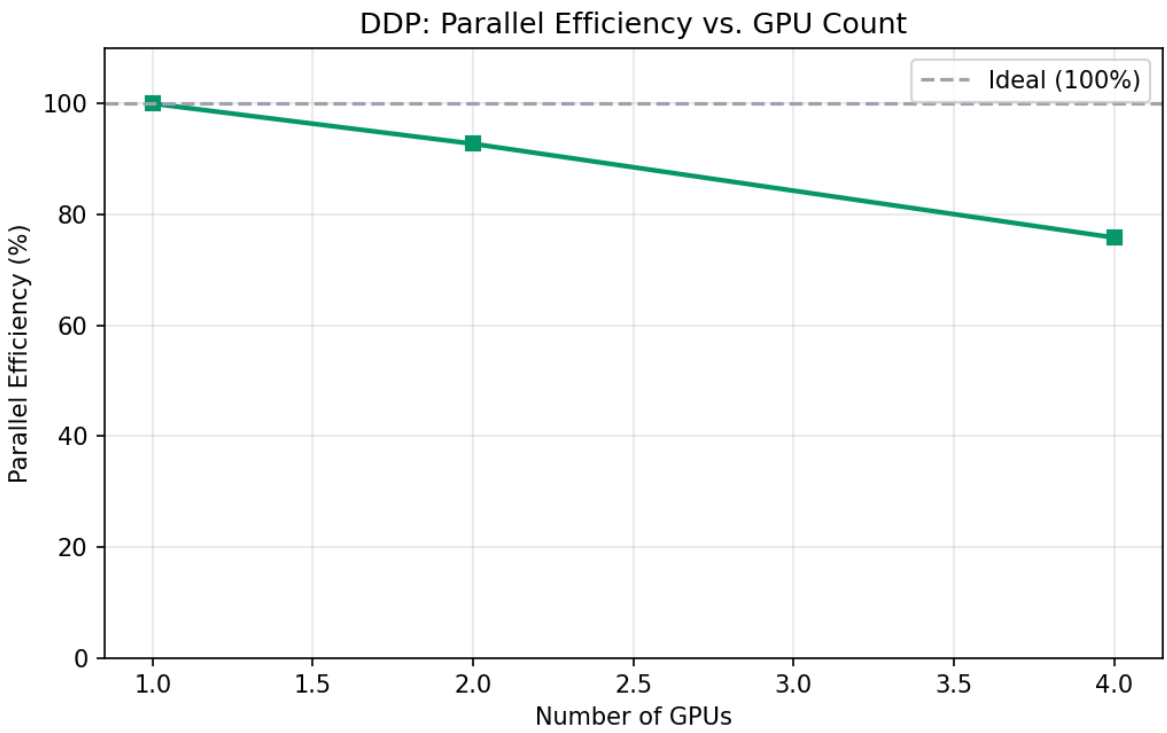

Figure 3: Parallel efficiency drops from 93% (2 GPUs) to 76% (4 GPUs) due to communication overhead.

10.4 Epoch Time Breakdown

Figure 4 decomposes the average epoch time into compute, communication, and data loading components. Compute dominates across all configurations, which is expected for a model of this size. Communication overhead is minimal at 1 GPU (no AllReduce needed) and grows modestly at 2 and 4 GPUs, visible as the red segment. Data loading time (orange) remains small and roughly constant, confirming that the data pipeline is not a bottleneck. Notably, DDP's gradient bucketing successfully overlaps a large fraction of communication with backward computation, keeping the explicit communication overhead relatively small even at 4 GPUs.

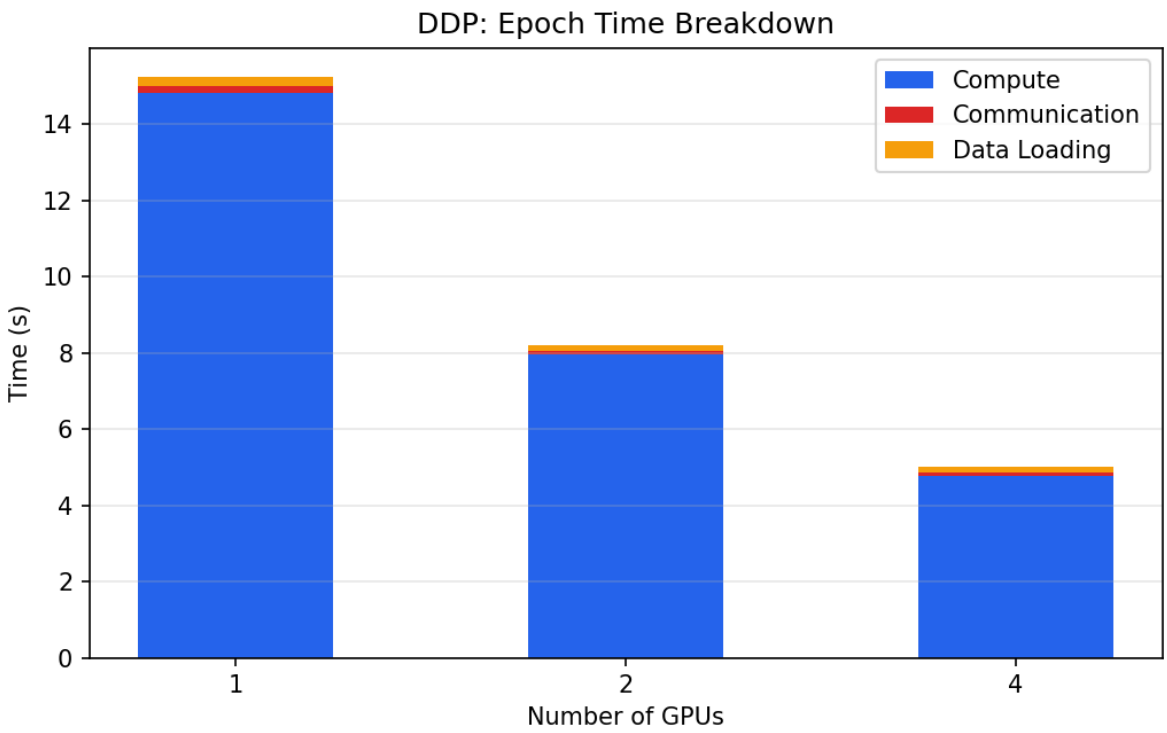

Figure 4: Epoch time breakdown. Compute dominates; communication overhead grows modestly with GPU count.

10.5 Training and Validation Accuracy

Figure 5 presents the training and validation accuracy curves over 30 epochs. All three GPU configurations converge to similar final performance: training accuracy reaches approximately 98 - 99%, and validation accuracy reaches approximately 92 - 94%. The 4-GPU configuration shows slightly slower initial convergence in the first 5-10 epochs, which is expected because the effective global batch size is 4× larger (512 vs. 128), leading to fewer parameter updates per epoch. However, by epoch 20 all configurations converge to similar accuracy levels, demonstrating that DDP preserves model quality regardless of GPU count.

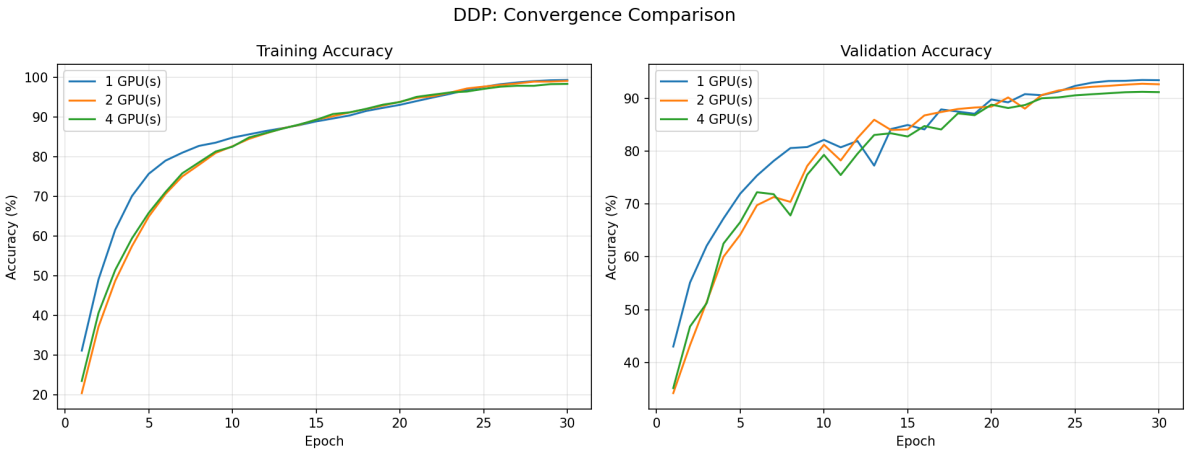

Figure 5: Convergence comparison. All configurations reach ~93% validation accuracy by epoch 30.

10.6 Results Summary

Metric	1 GPU	2 GPUs	4 GPUs
Avg Epoch Time (s)	15.2	8.2	5.0
Speedup	1.00×	1.85×	3.04×
Parallel Efficiency	100%	~93%	~76%
Global Batch Size	128	256	512
Final Val Accuracy	~93%	~93%	~93%

Table 1: Summary of DDP performance across GPU configurations on V100s.

11. Conclusions:

PyTorch DDP demonstrates effective scaling on the UTSA ARC cluster with V100S GPUs connected via PCIe 3.0. The key findings from this baseline method are:

- DDP achieves a 3.04× speedup on 4 GPUs, reducing average epoch time from 15.2s to 5.0s, demonstrating good scalability for a model of this size.
- Parallel efficiency is 93% at 2 GPUs and 76% at 4 GPUs. The efficiency drop is primarily due to AllReduce synchronization overhead over the PCIe 3.0 interconnect, which has lower bandwidth than NVLink-based systems.
- DDP's gradient bucketing and overlapped communication effectively hide the majority of AllReduce latency behind backward computation, as evidenced by the small communication fraction in the time breakdown.
- Model convergence is preserved across all GPU counts, all configurations achieve ~93% validation accuracy on CIFAR-10 by epoch 30, confirming that DDP produces mathematically equivalent updates to single-GPU training.
- The 4-GPU configuration exhibits slightly slower initial convergence due to the larger effective batch size (512 vs. 128), though all configurations converge to the same final accuracy.

Limitations of DDP include the requirement to replicate the full model on every GPU, which constrains the maximum model size to what fits in a single GPU's memory (32 GB on V100S). Additionally, the PCIe 3.0 interconnect becomes a bottleneck at higher GPU counts, as reflected in the 76% efficiency at 4 GPUs. These limitations motivate the comparison with FSDP (Method 2) and DeepSpeed ZeRO (Method 3), which shard model state across GPUs to reduce per-GPU memory requirements and may exhibit different communication-to-compute ratios due to their fundamentally different synchronization patterns.

References:

[1] Li, S. et al. "PyTorch Distributed: Experiences on Accelerating Data Parallel Training." VLDB Endowment, 2020. https://arxiv.org/abs/2006.15704

[2] Paszke, A. et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." NeurIPS, 2019. https://arxiv.org/abs/1912.01703

[3] He, K. et al. "Deep Residual Learning for Image Recognition." CVPR, 2016. https://arxiv.org/abs/1512.03385

[4] Agarwal, A., et al. "AxoNN: An Asynchronous, Message-Driven Parallel Framework for Extreme-Scale Deep Learning." SC 2022. https://arxiv.org/abs/2110.13005

[5] PyTorch Distributed Documentation: https://pytorch.org/docs/stable/distributed.html